University of Asia Pacific
Department of Computer Science and Engineering
CSE 402: Operating System Lab

Lab Report: 04

Report Title: SJF Preemptive

Submitted by:

Name : Adika Sultana

Student ID : 23101162

Section : C2

Submitted to:

Atia Rahman Orthi
Lecturer,
Department of CSE, UAP

Date of Submission: 27/08/2026

1. Problem Statement:

Consider the following set of processes with their Arrival Time (AT) and Burst Time (BT):

Process	Arrival Time (AT)	Burst Time (BT)
P1	4	2
P2	2	2
P3	1	3
P4	0	6
P5	3	1

Using a Preemptive SJF scheduling algorithm with a Time Quantum of 2 seconds, perform the following tasks:

1. Determine the Process Execution Sequence.
2. Calculate the Completion Time (CT**)** for each process.
3. Calculate the Turnaround Time (TAT) for each process.
4. Calculate the Waiting Time (WT) for each process.
5. Calculate the Average Turnaround Time (Average TAT).
6. Calculate the Average Waiting Time (Average WT).

2. Source Code:

```
processes = [
    ["P1", 4, 2],
    ["P2", 2, 2],
    ["P3", 1, 3],
    ["P4", 0, 6],
    ["P5", 3, 1]
]
Time_Quantum = 2
n = len(processes)
remaining = {}

for p in processes:
    remaining[p[0]] = p[2]
CT = {}
sequence = []
```

```
time = 0

while len(CT) < n:
available = []

    for p in processes:
        name = p[0]
        AT = p[1]

        if AT <= time and remaining[name] > 0:
            available.append(p)
    if len(available) == 0:
        time += 1
        continue

    current = min(
        available,
        key=lambda p: remaining[p[0]]
    )

    name = current[0]

    sequence.append(name)


    run_time = min(Time_Quantum, remaining[name])

    remaining[name] -= run_time
    time += run_time

    if remaining[name] == 0:
        CT[name] = time

TAT = {}
WT = {}

for p in processes:

    name = p[0]
    AT = p[1]
```

```
    BT = p[2]
    TAT[name] = CT[name] - AT
    WT[name] = TAT[name] - BT

print("Process Execution Sequence:")
print(" -> ".join(sequence))
print("\n")
print(f"{'P':<8}{'AT':<8}{'BT':<8}{'CT':<8}{'TAT':<8}{'WT':<8}")

total_tat = 0
total_wt = 0

for p in processes:

    name = p[0]
    AT = p[1]
    BT = p[2]

print(
        f"{name:<8}"
        f"{AT:<8}"
        f"{BT:<8}"
        f"{CT[name]:<8}"
        f"{TAT[name]:<8}"
        f"{WT[name]:<8}"
    )

    total_tat += TAT[name]
    total_wt += WT[name]

avg_tat = total_tat / n
avg_wt = total_wt / n
print("\n")
print(f"Average TAT = {avg_tat:.2f} sec")
print(f"Average WT  = {avg_wt:.2f} sec")
```

GitHub:
https://github.com/Adika2003/CSE-402-Operating-System-Lab

3.Result:

```
C:\Users\lab\PycharmProjects\PythonProject\.venv\Scripts\python.exe "C:
Process Execution Sequence:
P4 -> P2 -> P5 -> P1 -> P3 -> P3 -> P4 -> P4


P       AT      BT      CT      TAT     WT
P1      4       2       7       3       1
P2      2       2       4       2       0
P3      1       3       10      9       6
P4      0       6       14      14      8
P5      3       1       5       2       1


Average TAT = 6.00 sec
Average WT  = 3.20 sec

Process finished with exit code 0
```

4.“For the given problem, which scheduling approach is better—Preemptive or Non-Preemptive? Justify your answer.”

Ans: For the given problem, Preemptive Scheduling is better than Non-Preemptive Scheduling because it gives the CPU more flexibility to handle newly arrived shorter processes. In this problem, P4 arrives first at time 0 with a burst time of 6 seconds**,** which is much longer than some of the other processes. If we use Non-Preemptive SJF, once P4 starts, it must finish before another process can execute, even if a shorter process such as P5 with a burst time of only 1 second arrives. This can make shorter processes wait unnecessarily. In contrast, Preemptive SJF can stop the current process and execute a shorter process when appropriate. Therefore, shorter processes can finish earlier, which reduces their waiting and turnaround times. From our calculation, the Preemptive approach produces an Average Waiting Time of 3.2 seconds and an Average Turnaround Time of 6.0 seconds**,** while the Non-Preemptive approach gives higher values. Thus, because it provides lower waiting time, lower turnaround time, faster completion of shorter processes, and better responsiveness**,** Preemptive Scheduling is the more suitable approach for this particular problem.

5. Discussion:

In this problem, Preemptive SJF (Shortest Job First), also known as Shortest Remaining Time First (SRTF), is used to determine which process should get the CPU at each point in time. The main idea of SRTF is that the process having the shortest remaining burst time gets priority. Unlike Non-Preemptive SJF, a running process can be interrupted if another process arrives with a shorter remaining burst time. For example, P4 arrives at time 0 with a burst time of 6 seconds, but later processes such as P2 and P5 arrive with much shorter burst times. Therefore, the scheduler can give priority to these shorter processes instead of allowing a long process to occupy the CPU continuously. This helps shorter processes finish earlier and generally reduces their waiting time. For the given process set, the execution sequence is determined by comparing the remaining burst times whenever scheduling decisions are made. The calculated Average Turnaround Time is 6.0 seconds, while the Average Waiting Time is 3.0 seconds. These results indicate that SRTF can provide efficient scheduling for processes with different burst times. However, preemptive scheduling may cause additional context-switching overhead because the CPU can switch between processes multiple times. Overall, Preemptive SJF is suitable for this problem because it prioritizes shorter remaining processes, improves response for short jobs, and keeps the average waiting time relatively low.